

FreeRTOS基础（五）：任务挂起与恢复

原创  VIP免费 已于 2024-06-02 00:05:37 修改 · 2.9k 阅读 ·  43 ·  50

今天我们将探讨FreeRTOS 中的两个非常重要的函数：任务挂起和恢复函数。在实际的嵌入式系统 开发中，我们常常需要在特定条件下暂停执行，而在满足某些条件后再恢复这些任务的执行。这就像我们日常生活中的“暂停”和“继续”按钮。无论是为了节省资源，还是为了确保系统的稳定性作都是至关重要的。通过理解和掌握任务的挂起与恢复，我们可以更灵活和高效地管理系统中的任务，确保各个任务能够协调运作，从而实现更稳定系统表现。接下来，我们将深入了解如何使用这两个函数，以及它们在实际应用中的场景和注意事项。

目录

- 一、任务挂起与恢复函数
- 二、任务挂起函数
- 三、任务恢复函数（任务中恢复）
- 四、FreeRTOS 中断优先级概述
- 五、任务恢复函数（中断中恢复）
- 六、任务挂起与恢复API函数解析（选学）
 - 6.1 任务挂起函数vTaskSuspend()
 - 6.2 （任务中调用）任务恢复函数vTaskResume()
 - 6.3 （中断中调用）任务恢复函数xTaskResumeFromISR()
- 七、任务挂起与恢复实验
- 八、任务挂起与任务恢复函数的使用场景
 - 8.1 临界区保护
 - 8.2 资源管理
 - 8.3 任务间同步

一、任务挂起与恢复函数

API函数	描述
vTaskSuspend()	挂起任务
vTaskResume()	恢复被挂起的任务
xTaskResumeFromISR()	在中断中恢复被挂起的任务

挂起	挂起任务类似暂停，可恢复； 删除任务，无法恢复，类似 “人死
恢复	恢复被挂起的任务
“FromISR”	带FromISR后缀是在中断函数中专用的API函数

CSDN @未来可期，

二、任务挂起函数



使用前先使能API函数

void vTaskSuspend(TaskHandle_t xTaskToSuspend)

形参	描述
xTaskToSuspend	待挂起任务的任务句柄

此函数用于挂起任务，使用时需将宏 INCLUDE_vTaskSuspend 配置为 1。无论优先级如何，被挂起的任务都将不再被执行，直到任务被恢复。

注意：当传入的参数为NULL，则代表挂起任务自身（当前正在运行的任务）

三、任务恢复函数（任务中恢复）

使用前先使能API函数

任务中恢复被挂起函数：void vTaskResume(TaskHandle_t xTaskToResume)

形参	描述
xTaskToResume	待恢复任务的任务句柄

使用该函数注意宏：INCLUDE_vTaskSuspend必须定义为 1

注意：任务无论被 vTaskSuspend() 挂起多少次，只需在任务中调用 vTakResume() 恢复一次，继续运行。且被恢复的任务会进入就绪态！

CSDN @未来可期

四、FreeRTOS 中断优先级概述

FreeRTOS 的配置文件中 FreeRTOSConfig.h 中有两个关键宏定义：

cppAI写代码

```
1 configMAX_SYSCALL_INTERRUPT_PRIORITY //定义了可以使用 FreeRTOS API 的最高中断优先级。
2 configKERNEL_INTERRUPT_PRIORITY //定义了 FreeRTOS 内核的优先级
```

我们知道，Stm32一共有10个系统中断（内核中断），FreeRTOS便可以对这些系统中断进行管理；
第一个宏其实代表的是FreeRTOS可以管理的中断的最高的优先级；在 ARM Cortex-M 微控制器上，优先级数值越低，优先级越高。
configMAX_SYSCALL_INTERRUPT_PRIORITY 通常设置为一个中等优先级数值。例如，如果 FreeRTOS 管理的最高优先级为 5，则 configMAX_SYSCALL_INTERRUPT_PRIORITY 可以设置为 5。

解释：中断优先级与 FreeRTOS API
如果在中断服务程序（ISR）中调用 FreeRTOS 的 API 函数，该 ISR 的优先级必须小于或者等于 configMAX_SYSCALL_INTERRUPT_PRIORITY 意味着中断优先级数值必须大于等于 configMAX_SYSCALL_INTERRUPT_PRIORITY 的值。
如果 ISR 的优先级高于 configMAX_SYSCALL_INTERRUPT_PRIORITY（即数值低于 configMAX_SYSCALL_INTERRUPT_PRIORITY），在该 ISR 中调用 FreeRTOS API 函数可能会导致不可预期的行为，例如任务切换不正确或系统崩溃。这是因为 FreeRTOS 的调度器和其他 API 函数不是为在高优先级上下文中执行而设计的，即该中断不受FreeRTOS的管理。

cppAI写代码

```
1 #define configMAX_SYSCALL_INTERRUPT_PRIORITY 5
2
```

```
3 // 假设这是一个中断服务程序，优先级为6
4 void MyISR(void) {
5     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
6
7     // 恢复任务，只有优先级为6或更低的中断才能调用该函数
8     vTaskResumeFromISR(xTaskHandle);
9
10    // 如果恢复的任务优先级较高，可以请求上下文切换
11    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
12 }
```

五、任务恢复函数（中断中恢复）

使用前先使能API函数

中断中恢复被挂起函数： `BaseType_t xTaskResumeFromISR(TaskHandle_t xTaskToResume)`

形参	描述
<code>xTaskToResume</code>	待恢复任务的任务句柄

函数： `xTaskResumeFromISR` 返回值描述如下：

返回值	描述
<code>pdTRUE</code>	任务恢复后需要进行任务切换（恢复的任务的优先级大于正在运行的任务的优先级）
<code>pdFALSE</code>	任务恢复后不需要进行任务切换（恢复的任务的优先级小于或者等于正在运行的任务的优先级）

使用该函数注意宏： `INCLUDE_vTaskSuspend` 和 `INCLUDE_xTaskResumeFromISR` 必须定义为该函数专用于中断服务函数中，用于解挂被挂起任务

注意：中断服务程序中要调用freeRTOS的API函数则中断优先级不能高于FreeRTOS所管理的最高优先级（5-15）即中断优先级必须在5-15之内，中断优先级是越小，优先级越高。

CSDN @未来可期

六、任务挂起与恢复API 函数解析（选学）

6.1 任务挂起函数 `vTaskSuspend()`

1、任务挂起函数 `vTaskSuspend()`

1、需将宏 `INCLUDE_vTaskSuspend` 配置为 1

2、根据任务句柄获取任务控制块，如果任务句柄为 `NULL`，表示挂起任务自身

3、将要挂起的任务从相应的状态列表和事件列表中移除

4、将待挂起任务的任务状态列表项插入到挂起态任务列表末尾

5、判断任务调度器是否运行，在运行，更新下一次阻塞时间，防止被挂起任务；次阻塞超时任务

6、如果挂起的是任务自身，且调度器正在运行，需要进行一次任务切换；调度器没有运行，判断挂起任务数是否等于任务总数，是：当前控制块赋值为 `N` 否：寻找下一个最高优先级任务

CSDN @未来可期

6.2 （任务中调用）任务恢复函数 `vTaskResume()`

2、（任务中调用）任务恢复函数vTaskResume()

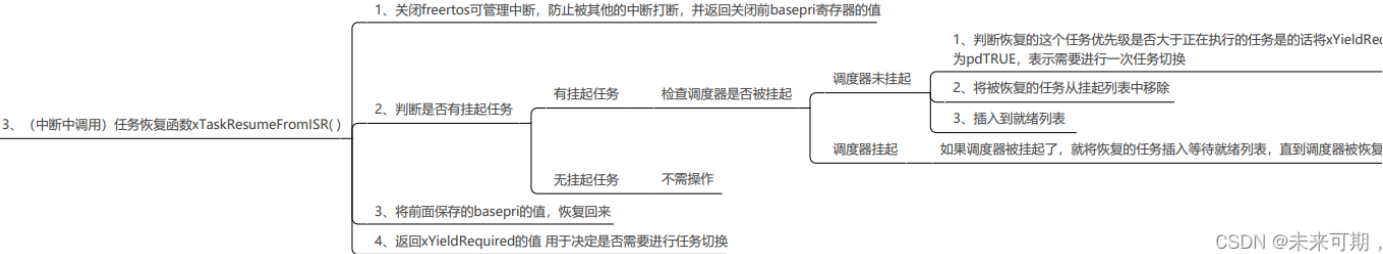
2、恢复任务不能是正在运行任务

3、判断任务是否在挂起列表中，是：就会将该任务在挂起列表中移除， 将该到就绪列表中

4、判断恢复的任务优先级是否大于当前正在运行的 是的话执行任务切换

CSDN @未来可期，

6.3（中断中调用）任务恢复函数xTaskResumeFromISR()



CSDN @未来可期，

七、任务挂起与恢复实验

1、实验目的：学会 使用FreeRTOS中的任务挂起与恢复相关API函数： vTaskSuspend(), vTaskResume(),

2、实验设计：将设计四个任务： start_task、 task1、 task2、 task3

四个任务的功能如下：

start_task 用来创建其他的三个任务

task1 实现LED0每500ms闪烁一次

task2 实现LED1每500ms闪烁一次

task3 判断按键按下逻辑，KEY0按下，挂起task1，按下KEY1在任务中恢复task1

CSDN @未来可期，静待花开~

创建任务及实现任务函数：

cpp

AI写代码

```
1  #include "stm32f4xx.h"           // Device header
2  #include "stdio.h"
3  #include "FreeRTOS.h"
4  #include "task.h"
5  #include "dynamic.h"
6
7
8
9  /*****START_TASK 任务配置*****/
10 /***** 包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
11
12 #define START_TASK_STACK_SIZE 128 //定义堆栈大小为128字 (1字等于4字节)
13 #define START_TASK_PRIO 1 //定义任务优先级, 0-31根据任务需求
14 TaskHandle_t start_task_handler; //定义任务句柄 (结构体指针)
15 void start_task(void* args);
16
17
18
19
20 /*****TASK1 任务配置*****/
21 /***** 包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
22 #define TASK1_STACK_SIZE 128 //定义堆栈大小为128字 (1字等于4字节)
23 #define TASK1_PRIO 2 //定义任务优先级, 0-31根据任务需求
24 TaskHandle_t task1_handler; //定义任务句柄 (结构体指针)
25 void task1(void* args);
26
27
28
```

```

29 | 30 | /*****TASK2任务配置*****/
31 | /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
32 | #define TASK2_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
33 | #define TASK2_PRIIO 3 //定义任务优先级，0-31根据任务需求
34 | TaskHandle_t task2_handler; //定义任务句柄（结构体指针）
35 | void task2(void* args);
36 |
37 |
38 |
39 | /*****TASK3任务配置*****/
40 | /*****包括任务堆栈大小、任务优先级、任务句柄、创建任务*****/
41 | #define TASK3_STACK_SIZE 128 //定义堆栈大小为128字（1字等于4字节）
42 | #define TASK3_PRIIO 4 //定义任务优先级，0-31根据任务需求
43 | TaskHandle_t task3_handler; //定义任务句柄（结构体指针）
44 | void task3(void* args);
45 |
46 |
47 |
48 |
49 | /*****开始任务用来创建其他三个任务，只创建一次，不能是死循环，同时创建完3个任务后删除任务1本身*****/
50 | void start_task(void* args)
51 | {
52 |     taskENTER_CRITICAL(); /*进入临界区*/
53 |
54 |     xTaskCreate( (TaskFunction_t) task1,
55 |                 (char *) "task1",
56 |                 ( configSTACK_DEPTH_TYPE) TASK1_STACK_SIZE,
57 |                 (void *) NULL,
58 |                 (UBaseType_t) TASK1_PRIIO ,
59 |                 (TaskHandle_t *) &task1_handler );
60 |
61 |     xTaskCreate( (TaskFunction_t) task2,
62 |                 (char *) "task2",
63 |                 ( configSTACK_DEPTH_TYPE) TASK2_STACK_SIZE,
64 |                 (void *) NULL,
65 |                 (UBaseType_t) TASK2_PRIIO ,
66 |                 (TaskHandle_t *) &task2_handler );
67 |
68 |     xTaskCreate( (TaskFunction_t) task3,
69 |                 (char *) "task3",
70 |                 ( configSTACK_DEPTH_TYPE) TASK3_STACK_SIZE,
71 |                 (void *) NULL,
72 |                 (UBaseType_t) TASK3_PRIIO ,
73 |                 (TaskHandle_t *) &task3_handler );
74 |
75 |     vTaskDelete(NULL); //删除开始任务自身，传参NULL
76 |
77 |     taskEXIT_CRITICAL(); /*退出临界区*/
78 |
79 |
80 |     //临界区内不会进行任务的调度切换，出了临界区才会进行任务调度，抢占式
81 | }
82 |
83 |
84 |
85 |
86 | /*****其余三个任务的任务函数，无返回值且是死循环*****/
87 |
88 | /***任务1：实现LED0每500ms翻转一次*****/
89 | void task1(void* args)
90 | {
91 |     uint32_t task1_num=0;
92 |     while(1)
93 |     {
94 |         printf("task1_num:%d\n",++task1_num);
95 |         GPIO_ToggleBits(GPIOF,GPIO_Pin_9 );
96 |         vTaskDelay(500); //FreeRTOS自带的延时函数
97 |
98 |     }
99 |
100 |

```

```

101 | }
102 |
103 | /** 任务2: 实现LED1 每500ms 翻转一次*****/
104 | void task2(void* args)
105 | {
106 |     uint32_t task2_num=0;
107 |     while(1)
108 |     {
109 |         printf("task2_num:%d\n", ++task2_num);
110 |         GPIO_ToggleBits(GPIOF, GPIO_Pin_10 );
111 |         vTaskDelay(500);      //FreeRTOS 自带的延时函数
112 |     }
113 |
114 |
115 | }
116 |
117 |
118 | /** 任务3: 判断按键KEY0, 按下KEY0, 任务1删除*****/
119 | void task3(void* args)
120 | {
121 |     while(1)
122 |     {
123 |
124 |         if(GPIO_ReadInputDataBit(GPIOE, GPIO_Pin_4)==0)  //表示按键Key0 按下
125 |         {
126 |             vTaskSuspend(task1_handler);                  //挂起任务1
127 |         }
128 |         else if (GPIO_ReadInputDataBit(GPIOE, GPIO_Pin_4)==0)  //表示按键Key1 按下
129 |         {
130 |             vTaskResume(task1_handler);                    //恢复任务1
131 |         }
132 |         vTaskDelay(10);
133 |     }
134 |
135 | }
136 |
137 |
138 |
139 |
140 |
141 | //FreeRTOS入口例程函数, 无参数, 无返回值
142 | void freertos_demo(void)
143 | {
144 |     xTaskCreate( (TaskFunction_t)    start_task,
145 |                 (char *)             "start_task",
146 |                 ( configSTACK_DEPTH_TYPE)  START_TASK_STACK_SIZE,
147 |                 (void *)             NULL,
148 |                 (UBaseType_t) START_TASK_PRIO ,
149 |                 (TaskHandle_t *)      &start_task_handler );
150 |
151 |
152 |     vTaskStartScheduler();  //开启任务调度器
153 |
154 | }

```

主函数调用入口函数，操作系统开始进行任务的切换和调度

cpp

AI写代码

```

1 | #include "stm32f4xx.h"           // Device header
2 | #include "stdio.h"
3 | #include "myled.h"
4 | #include "mykey.h"
5 | #include "myusart.h"
6 |
7 | #include "FreeRTOS.h"
8 | #include "task.h"
9 | #include "dynamic.h"
10 |

```

```
11 | int main(void)
    | 12 | {
13 |     // 硬件初始化
14 |     My_UsartInit();
15 |     LED_Init();
16 |     KEY_Init();
17 |
18 |     // 调用入口函数
19 |     freertos_demo();
20 |
21 |
22 |
23 | }
```

八、任务挂起与任务恢复函数的使用场景

8.1 .临界区保护

在某些情况下，需要确保一段代码不被中断或其他任务打断。例如，在对共享资源进行操作时，可以使用任务挂起和恢复函数来实现临界区保护

void vTaskSuspendAll(void)
可将所有任务挂起，实质是将任务调度器锁定。

cpp

AI写代码

```
1 | // 共享资源操作示例
2 | void critical_section(void) {
3 |     // 挂起其他任务
4 |     vTaskSuspendAll();
5 |
6 |     // 临界区代码
7 |     shared_resource++;
8 |
9 |     // 恢复任务调度
10 |    xTaskResumeAll();
11 | }
```

8.2 资源管理

在访问某些硬件资源（如SPI 总线、I2C总线等）时，需要**确保在使用资源期间不会被其他任务干扰。这时可以挂起其他可能访问该资源的任**
毕后再恢复它们。

cpp

AI写代码

```
1 | void access_resource_task(void *pvParameters) {
2 |     // 挂起可能使用相同资源的任务
3 |     vTaskSuspend(task_handle);
4 |
5 |     // 操作硬件资源
6 |     use_hardware_resource();
7 |
8 |     // 恢复任务
9 |     vTaskResume(task_handle);
10 | }
```

8.3 任务间同步

在某些场景下，可能**需要一个任务等待另一个任务完成特定操作后再继续执行。**可以通过挂起和恢复任务实现这种同步机制。

cpp

AI写代码

```
1 | // 任务A
2 | void taskA(void *pvParameters) {
3 |     // 执行某些操作
4 |
5 |     // 挂起任务A自己
6 |     vTaskSuspend(NULL);
7 | }
```

```
8 | // 当任务B完成操作后任务A将被恢复
9 | }
10 |
11 | // 任务B
12 | void taskB(void *pvParameters) {
13 |     // 执行任务A需要等待的操作
14 |
15 |     // 恢复任务A
16 |     vTaskResume(task_handleA);
17 | }
```

需要注意的是，滥用任务挂起和恢复函数可能导致系统响应变慢或引入难以调试的错误。在实际应用中，应尽量通过其他FreeRTOS提供的同步号量、消息队列 等）来实现任务间的协调和同步。

至此，任务挂起与恢复函数就已经讲解完毕！初次学习，循序渐进，一步步掌握即可！以上就是全部内容！请务必掌握，创作不易，欢迎大家点赞加您的支持是我前进最大的动力！下期再见！

标签 [#FreeRTOS](#)